Methods for Parsing Dipmeter Images

Bryan Azbill

August 2023

1 Introduction

The angle of rotation from horizontal along a layer of rock is called its dip. Wells typically pass through many layers of rock. Interests in measuring the dip of layer for such wells can be served by a device called the dipmeter, and it is common practice to store data collected with a dipmeter in a format referred to as the arrow or the tadpole plot. Dip logs for many wells have been stored as tadpole plots. Tadpole plots contain a cartesian grid to record each measurement over with the placement of a circle and juxtaposed perpendicular line, loosely in the shape of a tadpole, similar to a ♂. These shapes record the layers depth with their horizontal position, dip by their vertical position, These plots became common practice because they are convienently readable by human eyes, but when they are the only record available from a previous dip survey for a well, and those records are desired for a computational study, it is problematic to parse digital images of tadpole plots into numerical data for analysis. These images are often scanned in poor condition, the gridlines unevenly spaced and faded, the tadpoles distorted, and the images are not only crooked wavering in their angle. Having lines superimposed over the shapes makes distinquishing them particularly difficult, like captcha images. Granted, the images are not as bad as the captcha, but the problem is similar. Even though very effective algorithms for image parsing the likes of captcha systems have been already implemented with convolutional deep learning, they are not directly applicable to this problem because they rely on the fact that images are generated by a computer enable the generation of arbitrarily large training set images with known answers. The more traditional approaches used in this study circumvent the need for training sets by using a priori knowledge of the tadpole image to manually structure kernals for convolution and other filtering operations in order to overcome the above mentioned challenges and reliably extricate the different components superimposed in the image. The problem is broken down as follows,

1. extricating a grid from the rest of an image when

   the grid is located among other parts of the log

   the grid is not straight, even at an angle

the grid is not perfectly spaced

the grid fades out in places

the grid is interspaced with other tadpoles

2. extricating the tadpole circles placed upon the grid image when,

   the image is distorted

   clearing the grid from the circles adds further distortion

   some circles are solid while others are not

   resolution may be low

3. comparing the pixel location of each tadpole to that of the gridlines to determine the depth and dip of each measurement when

   the grid is not perfectly spaced

4. extricating the angle of the line tailed on each tadpole, when

   clearing both the circles and gridlines from the image prevents their interference introduces but iintroduces further distortion

   some lines intersect other circles and gridlines

The tadpoles interfere with detection on the grid. The grid interferes with detection on the tadpoles. both subset only a portion of the larger log image. the stricter a filter must be to remove the interference, the more undesired interference there will to the structure the filter is trying to isolate, so what gives? It stands to reason that as more information already collected on the image, the more strategically operations can be applied. The trick is bootstrapping an alternation series of algorithms which filter, algorithms which restore, and algorithms which detect. A series to use previous detections for progressively balancing the next round with less selective and distorive filters, applying both filters and detection at more strategic subsets of the image.

2 Methods

The following order structures the application of filters, detections, and restorations to the image:

1. identifying the location of the grid
2. restoring the image to a straightened grid
3. identifying the line spacing of the grid
4. identifying the locations of each tadpole
5. identifying the angle of the line extending each tadpole

6. converting each tadpoles pixel location to its placement relative to the cartesian grid

The filters chosen in this study are:

Convolutional Filters With kernals manually designed, they can be highly selective for isolating structures chosen for detection at the beginning of the process, before much else can be used to cleared other parts of the image or used for guidance on where to look.

With kernals selected from regions of the image, they can be used to compare the rest of the image with the selected region

Morphological Erode Is effective at clearing previously detected structures from the image with minimal distortion and is used more towards the end of the process.

Morphological Open

The algorithms chosen for detection are:

Peak Detection either applied to the image directly, or to a list of sums of the image by row or column

Hough Transform to locate circles

Template Matching to detect the angle on the tadpole tails

The algorithms used which restore are:

shifting to straighten the image

An algorithm used which both filters and restores:

Threshold applied to faded structures to remove what's too faded, and blacken what's not.

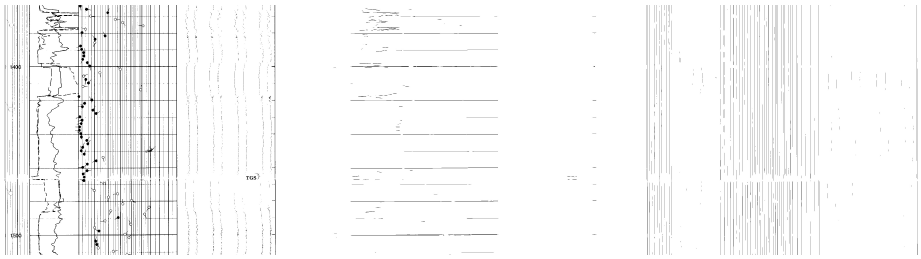
2.1 Identifying the location and spacing of the cartesian grid

Identifying the grid is the first step because it is the simplest to identify. This is because they can be distinguished by the fact that they are nearly vertical, horizontal, and also extend across large portions of the image.

The following kernal is used to identify vertical edges, it is called a Prewitt tranform, and compares to numerically computing a partial x derivative,

$$P = \begin{matrix} 1/3 & 1/3 & 1/3 \\ 0 & 0 & 0 \\ -1/3 & -1/3 & -1/3 \end{matrix} \tag{1}$$

To be used on horizontal lines, it simply needs be transposed For lines, however, it is better to compute the second partial derivative, which is done by applying



(a) Original (b) With L_h figures (c) With L_v

Figure 1: Filtering Out Lines

the prewitt transform twice. Finally the line filter should favor lines that continue for some distance. To accomplish this, the image is horizontally blurred with one final 1xn matrix with elements uniformly as 1/n, where n defines the blur distance. I call it B_n . all together, a kernal L_h is computed to be

$$L_h = P * P * B_n \tag{2}$$

where * denotes a convolution product.

From L_h, a vertical line convolution filter can be constructed trivially by simply computing the transpose of L_h,

$$L_v = L_h^t \tag{3}$$

To further clarify the lines after applying these filters, it helps to process with a threshold transformation. Then the results are very effective. Consider the following images as examples,

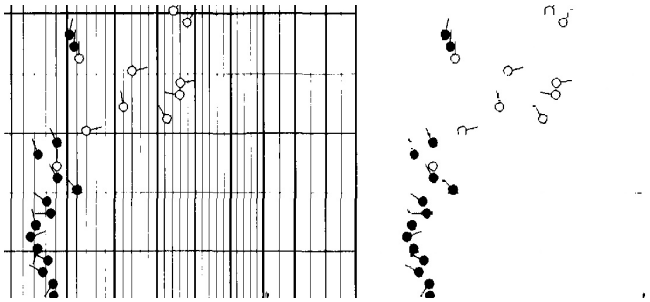
With the image filtered for lines, it is possible to identify the region between the top, bottom, left, and of the grid. by summing the filtered images row and column to find the peak where the lines are dense.

2.2 Straightening the Image and Detecting the Grid

Once the gridlines are detected, they can be used to detect shift in the image. Using the top of the grid filtered vertically as the kernal template for filtering the whole image, a convolution product is computed which reveals how the rows below can be shifted for their lines to match the top of the kernal. In a similar process, the column shifts are likewise identified. once this distortion is identified, it can be corrected by shifting the image.

2.3 Detecting The Tadpole Circles

When it comes to identifying the circles in an image, the Hough Transform is a standard and effective algorithm. However, it cannot be applied directly to



(a) a cut of the straightened image before clearing

(b) a cut of the straightened image after clearing

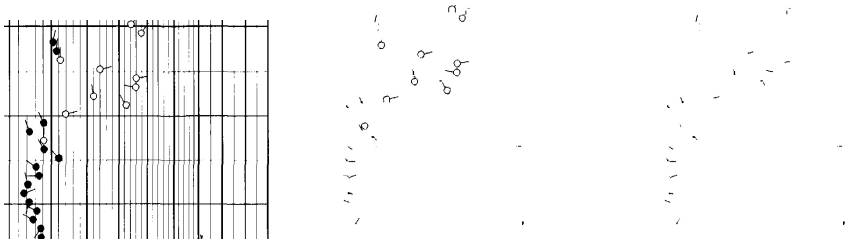
Figure 2: Filtering Out Lines

the image, because the gridlines will interfere with the detection. Therefore the gridlines must be cleared from the straightened image. Simply coloring them white distorts the circles. This is where morphological operations are useful. The morphological erode operation can be structured to whiten out the boundaries of every shape a chosen distance in either the vertical or horizontal direction. Such a distance can be chosen to be just enough to erase a whole gridline. Then, the morphological open operation is applied to invert the erosion, restoring any shape so long as it was not completely erased during the erosion. The combination of these two operations is called a morphological close, and is very effective at clearing the lines from the image while leaving minimal distortion to the circles. Additionally, the operation is only applied to the regions around each gridline. The following image shows the result of applying the morphological close operation in such a manner

Without nothing else to interfere, param Hough Transform can be applied to the image to detect where the tadpoles are placed with high sensitivity. In the first round of detection however some circles can be missed. Therefore, the first circles to be detected are erased from the image by simply drawing them white, and then the Hough Transform is applied again to detect the remaining circles. The following image shows the result of this process.

2.4 Processing the Tadpoles

In order for the tadpoles to be accurately identified, its necessary to mask out the underlying grid, so as not to confuse the gridlines with tadpoles. Once only the tadpoles remain with image, running a Hough transform algorithm out of a package is very effective in locating each circle, as well as computing their radius. Some tadpole circles are filled, while others are not. It's simple enough just to add up some of the pixels in the middle to determine the case for each one. Then it needs to be double checked if its really a circle. Doing this can be simply done by drawing another circle over the top and checking the difference. Once a tadpole is confirmed, the angle of the line extending from it needs to



(a) Original

(b) With First detections removed

(c) With Second detections removed

Figure 3: Filtering Out Lines

be determined. It is easy to do this by masking the circle from the tadpole and then using another out of the package algorithm. In some cases, however, the tadpole line is overlapped with a gridline, and so gets masked with the gridline. Therefore, lines with a poor match should be listed, so that at least the user of the script can identify

3 results

to be continued

4 Conclusion

The algorithms for this project were tested in python, which can be rather slow. Algorithms were tested from two libraries, OpenCV and Scikit-image. The OpenCV algorithms are compiled in C and just wrapped in python, and so therefore run very fast. Convolutions, hough transforms, and morphological operations where all best processed with OpenCV. Scikit-image is typically much slower, because it is not compiled in C. Scikit-image, however, is often easier to use and has a more comprehensive collection of tools with it. It's rather impressive. However, even after a great deal of time spent exploring the package and running tests with them, none provided better accuracy than effectively using the classic algorithms in OpenCV. Nonetheless, I found that often, major downsampling the image first can be done to improve runtime without compromising tests, and even though OpenCV bears the load of my final ideas for convolution kernal design that were key to finally cracking the whole problem were inspired by my study and experimentation with scikit-image.